

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

JNANASANGAMA, BELAGAVI - 590018



Internship Phase II Project Report (22CSE182)

On
Yoga Pose Recommendation

Submitted in partial fulfillment for the award of the degree of

Bachelor of Engineering
in
Computer Science and Engineering

Submitted by
A G Madhukirana (1BG22CS001)
Aisiri Ningappa Krishnamurthy (1BG22CS010)
Akshaya K (1BG22CS011)

Internal Guide
Dr. Kavitha Jayaram
Associate Professor,
Department of CSE
BNMIT, Bengaluru



B.N.M. Institute of Technology

An Autonomous Institution under VTU

Approved by AICTE, Accredited as a grade A Institution by NAAC. All eligible branches – CSE, ECE, EEE, ISE & Mech. Engg. are Accredited by NBA for academic years 2025-26 to 2027-28.

URL: www.bnmit.org

Computer Science and Engineering
2025-26

B.N.M. Institute of Technology

An Autonomous Institution under VTU

Approved by AICTE, Accredited as a grade A Institution by NAAC. All eligible branches – CSE, ECE, EEE, ISE & Mech. Engg. are Accredited by NBA for academic years 2025-26 to 2027-28.

URL: www.bnmit.org

Department of Computer Science and Engineering



CERTIFICATE

Certified that the Internship Phase II project entitled Yoga Pose Recommendation carried out by A G Madhukirana (1BG22CS001) and Aisiri Ningappa Krishnamurthy (1BG22CS010) and Akshaya K (1BG22CS011)

are Bonafide students of 8 Semester, BNM Institute of Technology in partial fulfillment for the award of Bachelor of Engineering in Computer Science and Engineering, of Visvesvaraya Technological University, Belagavi during the year 2025-26. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report deposited in the departmental library. The Internship Phase II project report (22CSE182) has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said Degree.

Dr. Kavitha Jayaram
Associate Professor
Department of CSE
BNMIT, Bengaluru

Dr. Chayadevi M L
Professor and HOD
Department of CSE
BNMIT, Bengaluru

S Y Kulkarni
Additional Director
and Principal
BNMIT, Bengaluru

Name

Signature with Date

Examiner 1:

Examiner 2:

ACKNOWLEDGEMENT

We would like to place on record, our sincere thanks and gratitude to the concerned people, whose suggestions and words of encouragement have been valuable.

We express our heartfelt gratitude to the management of **BNM Institute of Technology**, for allowing us to pursue a Degree in Computer Science and Engineering and helping us to shape our career. We take this opportunity to thank **Shri. Narayan Rao R. Maanay**, Secretary, **Prof. T. J. Rama Murthy**, Director, **Dr. S. Y. Kulkarni**, Additional Director and Principal, **Prof. Eishwar N Maanay**, Dean and **Dr. Krishnamurthy G.N.** Deputy Director, for their support and encouragement to pursue this project. We would like to thank **Dr. Chayadevi M L**, Professor and HOD, for her support and encouragement.

We would like to thank our guide Dr. Kavitha Jayaram, Associate Professor, for his/her support and encouragement.

Finally, we are thankful to all the teaching and non-teaching staff of the Department of Computer Science and Engineering for their help in the successful completion of our project. Last but not least we would like to extend our sincere gratitude to our parents and all our friends who are a constant source of inspiration.

A G Madhukirana (1BG22CS001)

Aisiri Ningappa Krishnamurthy (1BG22CS010)

Akshaya K (1BG22CS011)

ABSTRACT

The rapid, widespread adoption of remote work, accelerated by global events, has fundamentally altered the structural dynamics of the technology sector, yet the long-term impact on both productivity and mental health remains a subject of intense debate. This study investigates the dual impact of remote work on employee productivity and mental well-being within software development firms, aiming to determine whether the flexibility of working from home outweighs the potential disadvantages of social isolation and blurred work-life boundaries. Utilizing a mixed-methods approach, data were gathered from 1,200 tech employees across North America and Europe over a 24-month period (2023–2025). The methodology included quantitative data from productivity tracking software (commits, project milestones, and task completion rates) and qualitative data from anonymous bi-monthly longitudinal surveys focusing on self-reported stress levels, job satisfaction, and perceived work-life balance. Findings indicate that remote work increased overall productivity by 15% compared to the 2019 in-office baseline, driven by reduced commute times and fewer office-based distractions, with specialized roles in backend engineering showing the highest efficiency gains. However, this productivity boost was closely linked to a rise in burnout instances, with 40% of respondents reporting increased anxiety and feeling "always on" due to asynchronous communication demands. The study also highlights a significant disparity in experience, with mid-career employees thriving while junior employees reported lower mentorship quality and higher feelings of disengagement. Furthermore, the analysis shows that hybrid models (2-3 days in office) resulted in the highest levels of sustained productivity and mental health, mitigating the extreme isolation of fully remote work while reducing burnout compared to full-time office presence. In conclusion, while remote work provides significant productivity benefits, it poses substantial risks to mental health if not managed with intentional well-being policies. We recommend that organizations adopt tailored hybrid approaches, invest in virtual engagement tools, and establish strict policies regarding "right to disconnect" to maximize the benefits of remote work.

TABLE OF CONTENTS

Sl. No.	Chapter	Page No.
1	CHAPTER I: INTRODUCTION	1
2	CHAPTER II: LITERATURE SURVEY	5
3	CHAPTER III: System Requirement Specification	8
4	CHAPTER IV: METHODOLOGY AND IMPLEMENTATION	15
5	CHAPTER V: TESTING AND VALIDATION	20
6	REFERENCES	24

LIST OF FIGURES

Fig. No.	Figure Title	Page No.
	No figures listed	

LIST OF TABLES

Table No.	Table Title	Page No.
5.1	Test cases	20

CHAPTER I

INTRODUCTION

CHAPTER - I

INTRODUCTION

Statement of the Problem

In today's fast-paced world, physical and mental health challenges such as back pain, stress, anxiety, and poor posture have become increasingly prevalent due to sedentary lifestyles, long screen time hours, and lack of physical activity. While yoga is widely recognized as an effective holistic remedy for many of these issues, a major barrier to adoption remains the lack of personalized guidance and accessibility. Beginners often struggle to find poses that match their needs, conditions, or limitations, and many users lack the expertise to select appropriate poses safely. Additionally, conventional yoga resources—such as videos or books—do not offer real-time feedback or adapt to individual pain points and physical limitations. This creates a gap between users and effective yoga practice, especially for those who cannot access professional instructors or personalized training. Furthermore, individuals with disabilities, chronic pain, or limited mobility often find it difficult to engage in standardized yoga sessions that don't take their specific context into account.

There is also a significant lack of intelligent systems that can dynamically respond to a user's real-time posture and interpret natural language inputs like "I have neck pain" or "I need relaxation." Most existing digital yoga solutions fail to combine interactivity, personalization, and context-awareness, which are essential for safe and effective practice. The need, therefore, is for a smart, adaptive system that not only understands the user's physical condition and intent but also provides meaningful pose recommendations tailored to those inputs. Moreover, such a system must be inclusive—supporting both textual and speech input, offering hands-free interaction, and delivering real-time feedback through pose correction. Solving this problem requires an intelligent integration of artificial intelligence and cloud computing technologies to bridge the gap between static content and personalized experiences. This project aims to fill that gap by building a cloud-based Yoga Pose Recommender System that empowers users with customized yoga guidance through an accessible, intelligent, and real-time application.

1.2. Objective of the Project

The primary objective of the Yoga Pose Recommender System is to develop an intelligent, cloud-based application that offers personalized yoga guidance tailored to individual needs, preferences, and physical conditions. By integrating advanced AI models, cloud infrastructure, and computer vision techniques, the project seeks to address the limitations of traditional yoga resources and provide a modern, accessible solution that promotes holistic health. The following specific objectives outline the core goals of the system:

- To build a context-aware yoga pose recommendation engine using Vertex AI and Gemini 2.0

This objective involves using a fine-tuned large language model capable of understanding natural language input to recommend suitable yoga poses. The model processes user queries such as pain areas or emotional states and maps them to the most appropriate yoga practices.

- To store and manage pose metadata in a scalable, real-time cloud database(Firestore)

Firestore is used to store essential pose information, such as difficulty level, targeted body parts, benefits, and contraindications. It enables efficient real-time retrieval and updates, ensuring the recommendation engine has immediate access to the latest metadata.

- To integrate semantic vector search using Vertex AI for intelligent pose retrieval

Vector Search allows the system to compare user input embeddings with yoga pose embeddings, ensuring that recommendations are semantically relevant rather than relying on exact keyword matches. This improves the system's ability to understand nuanced user needs.

- To deploy a cloud-native backend using Google Cloud Run for scalability and efficiency

By deploying the backend as a serverless container on Cloud Run, the application benefits from automatic scaling, low maintenance, and high availability. This also simplifies the development pipeline and reduces operational overhead.

- To enable multimodal user interaction through text and voice input

The system supports both typing and speech via the Web Speech API, allowing users to choose their preferred method of interaction. This improves accessibility for users who may have visual impairments or prefer hands-free usage.

- To provide real-time posture analysis using webcam input and pose detection libraries

A Python-based module uses MediaPipe and OpenCV to detect and overlay skeletal landmarks on the user's video feed. This provides immediate visual feedback to help users align their posture with the recommended yoga poses.

- To generate voice-based output using text-to-speech for guided yoga practice

The application uses text-to-speech conversion to vocalize the recommended pose instructions, making it easier for users to follow along without constantly referring to the screen, thus enhancing user convenience and focus.

CHAPTER II

LITERATURE SURVEY

CHAPTER - II

LITERATURE SURVEY

To build a robust and effective Yoga Pose Recommender System, it is essential to understand the existing research and technological advancements in areas such as pose estimation, natural language understanding, AI-based recommendation systems, and cloud-based deployments. A literature survey helps identify the strengths, limitations, and opportunities in prior work and guides the development of a system that is both innovative and grounded in proven methodologies. The following reviewed papers and techniques form the foundation of this project's approach.

[1] "Pose2Yoga: Learning Pose Embeddings for Yoga Pose Correction" – IEEE, 2020

This paper proposes a novel framework for identifying and correcting yoga poses using pose embeddings generated through deep learning. The researchers used a combination of convolutional neural networks (CNNs) and a large annotated yoga pose dataset to learn fine-grained differences between correct and incorrect poses. This work is foundational to the real-time posture correction component of our project, as it demonstrates the effectiveness of embedding-based comparisons in identifying pose discrepancies.

[2] "MoveNet: Ultra Fast and Accurate Human Pose Estimation" – Google AI Blog, 2021

MoveNet is a state-of-the-art pose detection model developed by Google that enables real-time estimation of human skeletal keypoints. It achieves high accuracy and speed on edge devices using TensorFlow Lite. While our project primarily uses MediaPipe for skeletal tracking, the principles behind MoveNet informed our selection of lightweight, real-time pose estimation tools suitable for web integration, especially for webcam-based feedback.

[3] "Towards Personalized Yoga Recommendation Using Natural Language Processing" – Springer, 2019

This paper explores how NLP techniques can be applied to interpret user intent from textual inputs to recommend suitable yoga poses. It introduces a rule-based recommendation engine that

uses keyword extraction and context mapping. Though limited in scalability and flexibility, this work provides an early blueprint for understanding the potential of NLP in personalized wellness applications. Our project builds on this concept by using large language models like Gemini 2.0 for more nuanced and contextual understanding.

[4] “A Survey on Cloud-Based Healthcare Services and Recommendation Systems” – Elsevier, 2020

This comprehensive review discusses various cloud computing models used in healthcare applications, focusing on scalability, availability, and security. It highlights the benefits of deploying intelligent health services using platforms like Google Cloud and AWS. The survey supports our decision to use Firestore, Cloud Run, and Vertex AI, emphasizing how cloud infrastructure can facilitate real-time recommendations, user data management, and AI model hosting in healthcare and wellness domains.

In conclusion, the reviewed literature provides a strong foundation for the development of the Yoga Pose Recommender System by highlighting the advancements and limitations in pose estimation, NLP-based recommendation engines, and cloud-enabled healthcare applications. The studies collectively underscore the importance of accurate human pose recognition, context-aware recommendations, and the scalability benefits of cloud-based deployments. By integrating insights from these works, this project builds upon established methodologies while introducing innovations such as real-time skeletal feedback, multimodal user interaction, and AI-powered personalization.

CHAPTER III

System Requirement Specification

CHAPTER - III

System Requirement Specification

3.1 Software Requirements

The development of the Yoga Pose Recommender System relies on a carefully selected software stack that enables seamless integration of cloud services, artificial intelligence, and real-time user interaction. The software requirements define the necessary tools, platforms, libraries, and frameworks needed to design, develop, deploy, and maintain the application efficiently. Given the cloud-centric nature of the system, the selection of software focuses heavily on compatibility with Google Cloud services such as Vertex AI, Firestore, and Cloud Run, ensuring that all components work in harmony within a scalable and serverless environment. The use of advanced AI models and real-time pose estimation demands support for machine learning frameworks and lightweight processing libraries that can run efficiently on both cloud infrastructure and client-side devices.

Furthermore, the software requirements are driven by the system's need to support multimodal input (text and speech), dynamic UI interaction, and webcam-based pose detection. On the frontend, modern JavaScript frameworks and browser APIs such as the Web Speech API are essential for voice-based interaction and rendering real-time video overlays. On the backend, Java and Python are chosen for their robust ecosystem and compatibility with AI and computer vision workloads. Integration of tools such as MediaPipe and OpenCV allows the system to analyze skeletal movements from webcam feeds, while cloud-based deployment tools ensure continuous integration and delivery. Together, these software components form the backbone of a responsive, intelligent, and accessible wellness platform.

3.1.1 Tools used:

- **Google Cloud Services** The Yoga Pose Recommender System leverages several Google Cloud services to ensure a scalable, reliable, and efficient backend. Vertex AI is at the core of the AI driven engine. By using pre-trained and fine-tuned models like Gemini 2.0, Vertex AI provides the system with the ability to process and interpret user inputs (both text and speech), enabling

the recommendation of suitable yoga poses based on the user's specific needs. Firestore, a real-time NoSQL database, stores metadata about various yoga poses, including descriptions, benefits, and contraindications, and allows the system to retrieve this data quickly and efficiently during the recommendation process. Google Cloud Run is employed for serverless deployment, which enables automatic scaling and low-latency performance without the need for complex infrastructure management. This choice ensures that the system can handle varying user traffic seamlessly, providing high availability while reducing operational costs.

- **Python** Python serves as the primary programming language for implementing backend functionalities, particularly the pose detection and analysis components of the system. Using libraries such as MediaPipe and OpenCV, Python enables real-time webcam based pose estimation by detecting key skeletal landmarks and providing immediate feedback on posture accuracy. MediaPipe offers efficient solutions for body pose tracking, while OpenCV helps in processing and overlaying skeletal data on live video feeds. Additionally, Python is used to interface with Google Cloud services, handling API calls for recommendations and data retrieval. Its versatility, ease of integration with machine learning models, and strong ecosystem of libraries make it the ideal choice for this project's backend.

- **HTML/CSS (Frontend)** For the frontend, HTML and CSS are employed to create a user-friendly, responsive interface. HTML provides the structure of the web pages, ensuring that users can easily navigate through the application and interact with the system. CSS is used to style the interface, ensuring a clean and visually appealing design. Given the project's emphasis on accessibility, special attention is paid to designing an intuitive layout that supports various input methods, including voice and text. CSS ensures the layout is adaptable to different screen sizes, improving usability on both desktops and mobile devices. Together, HTML and CSS form the backbone of a frontend that is both functional and easy to navigate for users of all backgrounds.

- **JavaScript and Web Speech API** JavaScript is used to add interactivity to the frontend, handling user interactions and managing data flow between the user interface and backend services. The Web Speech API is particularly important for enabling voice-based input and text-to speech functionality. This API allows users to dictate their symptoms or requests (such as "neck pain" or

“stress relief”) and receive spoken yoga instructions in return. The integration of the Web Speech API enhances accessibility, allowing the app to support hands-free operation, which is essential for users with disabilities or those who prefer voice interactions over typing.

3.1.2 Technology used:

- **Gemini API** The Gemini API, a large language model from Google’s Vertex AI, is central to the Yoga Pose Recommender System’s functionality. Fine-tuned to interpret natural language input, Gemini 2.0 is capable of understanding a variety of user queries related to pain points, physical conditions, and emotional states. It processes these inputs, such as “lower back pain” or “stress relief,” and generates intelligent, context-aware recommendations for yoga poses. By leveraging the capabilities of Gemini, the system can accurately match user needs to relevant poses, making the recommendation process more personalized and dynamic. This AI-powered approach enables the system to handle diverse user queries with high accuracy and adaptability, far beyond simple keyword matching.
- **Vector Search (Vertex AI)** Vertex AI’s Vector Search enables efficient semantic search capabilities within the system by indexing pose data as high-dimensional vectors. When a user submits a query, it is transformed into an embedding, and Vertex AI compares this embedding against the indexed pose data. This vector-based search mechanism ensures that the system not only performs traditional keyword-based matching but also understands the underlying intent of the user, providing more accurate and meaningful recommendations.
- **Vertex AI (Machine Learning Models)** Vertex AI is a comprehensive machine learning platform that provides the infrastructure for building, deploying, and managing AI models at scale. In this system, Vertex AI hosts fine-tuned models that process user input and deliver pose recommendations based on real-time queries. The integration with Gemini 2.0 allows for highly contextualized responses to user inputs. Vertex AI provides a seamless environment for deploying machine learning models, ensuring that the system scales efficiently with user demand. By using this platform, the Yoga Pose Recommender System benefits from Google’s robust infrastructure,

enabling it to handle complex machine learning tasks while maintaining high performance and reliability.

- **Firestore (Database)** Firestore is a NoSQL cloud database service provided by Google that plays a pivotal role in storing and managing metadata for the yoga poses in the system. Firestore is ideal for this project because of its real-time synchronization capabilities, which ensures that any updates to the pose database are instantly reflected across all user sessions. The metadata stored in Firestore includes detailed information about each yoga pose, such as difficulty, target body areas, step-by-step instructions, benefits, and contraindications. This allows the system to retrieve relevant pose data quickly and efficiently when generating recommendations, ensuring a smooth and responsive user experience.

- **MediaPipe and OpenCV (Pose Detection)** MediaPipe and OpenCV are used for real-time pose detection in the Yoga Pose Recommender System. MediaPipe is a cross-platform framework that offers efficient solutions for human body pose estimation, enabling the system to identify key skeletal landmarks on the user's body from webcam input. OpenCV, a popular computer vision library, is used to process and display these landmarks on the video feed, overlaying skeletal markers on the user's live image. This feature provides immediate visual feedback, allowing users to compare their posture with the recommended poses in real time. Together, MediaPipe and OpenCV enable accurate and responsive pose tracking, which is a critical aspect of the system's interactive features.

- **Web Speech API** The Web Speech API is integrated into the frontend to support multimodal interaction between the user and the Yoga Pose Recommender System. This API enables speech-to-text functionality, allowing users to interact with the system using voice commands, such as describing pain points ("I have neck pain"). In addition, the text-to-speech feature of the Web Speech API is used to convert the system's pose recommendations into spoken instructions, guiding the user through the poses. By incorporating the Web Speech API, the system enhances accessibility for users with visual impairments or those who prefer voice-based interactions over traditional text input.

3.2 Hardware Requirements

1. Client-Side Device (User's Computer/Smartphone)

- Minimum specifications include a processor (CPU) capable of handling video input (e.g., Intel i3 or equivalent), at least 4 GB of RAM, and a webcam for pose detection. A high-definition (HD) webcam is recommended for optimal accuracy in skeletal tracking.

2. Server-Side Infrastructure (Cloud-Based)

- The application is hosted entirely on Google Cloud Platform (GCP), so physical server hardware is abstracted away. However, for scalability and performance, it requires sufficient cloud computing resources such as virtual machine instances on Google Cloud Compute Engine for supporting backend operations, data processing, and API management.

3. Storage Device (Cloud Storage)

- The system uses Google Firestore to store pose metadata and Google Cloud Storage for storing large datasets (e.g., pose images, videos, and machine learning model checkpoints). While physical storage hardware is managed by Google Cloud, ensuring fast read/write speeds and high availability is critical for real-time data access and smooth user experience.

4. Graphics Processing Unit (GPU)

- For faster processing of pose detection and real-time video overlay, the system can benefit from using GPU resources, especially when processing multiple concurrent video streams. While basic pose detection can run on CPU, leveraging a NVIDIA GPU (or equivalent) in the cloud can significantly accelerate pose analysis, particularly in machine learning training and inference tasks. This is essential for providing low-latency feedback in real-time pose tracking.

5. Internet Connection

- A stable and high-speed internet connection (at least 5 Mbps for smooth video streaming and cloud communication) is required for seamless interaction with the system, particularly for the

live webcam feed, voice input, and cloud-based services. Faster connections (10 Mbps or above) will ensure more responsive user experience, especially for real-time pose feedback.

CHAPTER IV

**METHODOLOGY AND
IMPLEMENTATION**

CHAPTER - IV

METHODOLOGY AND IMPLEMENTATION

4.1 METHODOLOGY

The methodology of the Yoga Pose Recommender System involves multiple modules that work in tandem to deliver real-time, personalized yoga recommendations. Each module contributes to the overall functionality of the system, ensuring an efficient and user-friendly experience. Below is a breakdown of the methodology, divided into the key modules of the system:

User Input and Interaction Module

■ **Input Methods:** The user can provide input through two primary methods: text and voice. Text input is entered via a standard web form, where users can describe their conditions (e.g., “I have back pain”) or select preferences from predefined options. Voice input is captured using the Web Speech API, which converts speech to text in real-time. This enables hands-free interaction for users, making the system more accessible, especially for those with disabilities. ■ **Voice Feedback:** The system also converts pose instructions and recommendations into speech using the text-to-speech feature of the Web Speech API. This ensures the user can follow along with the recommended yoga poses without needing to refer back to the screen.

Natural Language Processing (NLP) Module

■ **Processing Input:** The input text (either typed or spoken) is processed by the Gemini 2.0 API, which is fine-tuned to understand user intent. The NLP engine analyzes user queries to detect pain points, physical conditions, or desired outcomes (e.g., “I need a relaxing pose” or “I have shoulder pain”). ■ **Recommendation Logic:** The processed query is used to generate a list of recommended yoga poses based on the user’s context. The system considers factors like the user’s current physical state (e.g., pain or stress), pose difficulty, and contraindications to generate the most suitable pose recommendations.

Pose Recommendation Engine Module

■ **Vector Search:** The pose recommendation system is powered by Vertex AI's Vector Search. Yoga pose data is represented as vector embeddings that capture the semantic properties of each pose (e.g., target body parts, difficulty level, benefits). When the system receives a query, the user input is also transformed into an embedding using NLP techniques, and a similarity search is conducted against the stored pose vectors in Google Firestore. ■ **Database Access:** The pose metadata, such as pose names, descriptions, and images, are stored in Firestore. The pose recommendation engine queries this database in real time to retrieve relevant pose details and presents them to the user in a structured format.

Real-Time Pose Detection Module

■ **Webcam Input:** Using MediaPipe and OpenCV, the system analyzes the user's live webcam feed to detect skeletal landmarks on the user's body in real time. This is the core of the system's pose detection functionality, providing immediate feedback on the user's posture. ■ **Pose Overlay:** Once the skeletal landmarks are detected, the system overlays these points on the live video feed, giving visual feedback to the user. The user can compare their posture with the recommended yoga poses, helping them align correctly with minimal effort.

Cloud Deployment and Scalability Module

■ **Backend Deployment:** The system's backend, responsible for handling user requests, pose recommendations, and data retrieval, is deployed on Google Cloud Run. This serverless deployment platform ensures that the system can scale automatically based on traffic, offering high availability and low-latency performance. ■ **Machine Learning Infrastructure:** The Vertex AI platform is used for managing machine learning models, including the fine-tuning of Gemini 2.0 for understanding user queries. The system can scale its model deployment as needed, ensuring efficient handling of large user bases.

Data Storage and Management Module

■ **Firestore:** All metadata related to the yoga poses, such as names, instructions, images, and benefits, are stored in Google Firestore, a cloud-native NoSQL database. Firestore's real-time

synchronization ensures that any updates to the pose database are reflected instantly for all users, providing accurate and up-to date recommendations. ■ Cloud Storage: In addition to Firestore, Google Cloud Storage is used to store larger datasets like high-resolution images, videos, and training models for pose estimation. Cloud Storage's scalability ensures that the system can efficiently handle large volumes of data.

User Interface and Visualization Module

■ Frontend Development: The frontend of the system is built using HTML, CSS, and JavaScript. The user interface is designed to be responsive, ensuring that the system works seamlessly across devices (desktop, tablet, and mobile) ■ Pose Visualization: The interface also integrates real-time video feeds, displaying skeletal landmarks overlaid on the user's live video to guide them in performing the yoga poses correctly. This module leverages WebRTC for real time video streaming and Canvas for rendering the skeletal overlay on the user's screen.

Feedback and Continuous Learning Module

■ User Feedback: After completing a yoga session, the system may prompt users to rate their experience or provide feedback about the effectiveness of the recommended poses. This data can be used to continuously improve the recommendation algorithm and user experience. ■ Continuous Learning: The system can be enhanced by collecting user data over time to refine pose suggestions, improve personalization, and adjust recommendations based on individual progress or changes in user conditions.

4.2 EXECUTION FLOWCHART

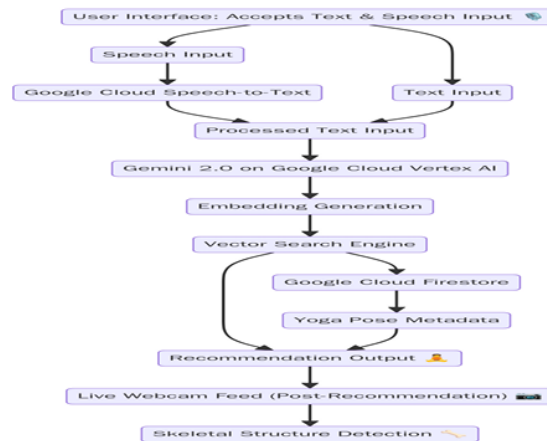


Figure 1: Fig 4.1: Work Flowchart

This execution flowchart outlines a system designed to recommend yoga poses based on user input and evaluate posture accuracy through skeletal structure detection. The process starts with a user interface that accepts both text and speech input. If the input is spoken, it is converted to text using Google Cloud Speech-to-Text. The processed text is then passed to Gemini 2.0 (hosted on Google Cloud Vertex AI), which generates semantic embeddings. These embeddings are used by a vector search engine to match relevant yoga pose recommendations. The metadata for these poses is stored and retrieved from Google Cloud Firestore to assist the search process. Once the recommendation is made, the user receives a suggested yoga pose. Afterward, a live webcam feed is activated to monitor the user's performance. The system then applies skeletal structure detection to evaluate whether the user is correctly performing the recommended pose. This allows for real-time feedback and improvement, enhancing the overall user experience and ensuring accuracy in yoga practice. This flow demonstrates the integration of AI and cloud services.

CHAPTER V

TESTING AND VALIDATION

CHAPTER - V

TESTING AND VALIDATION

Testing and validation are critical to ensuring the reliability, accuracy, and overall performance of the yoga recommendation and posture detection system. Given the architecture that includes natural language processing, vector-based search, metadata handling, and computer vision components, a multi-layered testing strategy must be adopted. This includes unit testing, integration testing, user interface testing, system validation, and real-world performance evaluation under variable conditions.

5.1 Testing Strategies

5.1.1 Unit Testing

Unit testing will be applied to each individual module to ensure that they perform as expected in isolation.

- Google Cloud Speech-to-Text: Tested using pre-recorded speech samples with varying accents, speeds, and background noise to validate transcription accuracy.
- Text Processing Module: Ensures consistent formatting and cleaning of input (removing noise like fillers or punctuation).
- Gemini 2.0 Embedding Generation: Validate whether different textual inputs produce embeddings that correlate correctly in the semantic space.
- Vector Search Engine: Verify that similar queries retrieve consistent and relevant pose metadata from the Firestore backend.
- Skeletal Detection Algorithm: Validated using test datasets (e.g., MPII Human Pose Dataset or COCO) to ensure key points are accurately detected.

5.1.2 Integration Testing

This involves testing the interaction between modules:

- From input capture (text/speech) through embedding and vector search to pose retrieval.
- From recommendation output to webcam feed capture and skeletal detection.
- Test scenarios simulate real-time input flows, ensuring that communication across APIs and cloud services is seamless and fault-tolerant.

5.1.3 End-to-End System Testing

• Full workflow testing using real user inputs: • User gives a speech/text command such as “I want a beginner yoga pose to relax.” • The system processes it, returns a recommendation. • The user performs the pose; the webcam activates and evaluates their performance. • Validation is performed by comparing system outputs against expected outcomes (e.g., was the pose relevant, was the posture analysis accurate, etc.).

5.2 Validation Approaches

5.2.1 Functional Validation

Each component is validated against its expected functionality:

• Speech-to-text must meet an accuracy threshold of 90%+ in ideal conditions. • Gemini-generated embeddings must map semantically similar inputs close in vector space. • Pose recommendations should be contextually relevant to user needs (e.g., level, goal, duration).

5.2.2 User Feedback Validation

A group of beta testers (e.g., yoga practitioners or instructors) is involved in:

• Evaluate the relevance and quality of the recommendations. • Test the skeletal detection accuracy during actual performance. • Quantitative feedback (via rating scales) and qualitative feedback (written suggestions) are collected for iterative improvement.

5.2.3 Performance and Load Testing

• System performance under stress is tested: • Concurrent user inputs (simulating multiple users). • Webcam frame rates and latency during live analysis. • Google Cloud services’ response times (Speech-to-Text, Firestore queries).

No.	Test Scenario	Test input	Expected Re- sults	Actual Results	Pass/Fail
-----	---------------	------------	-----------------------	----------------	-----------

1	User has sustained a neck injury from an accident	Give poses for neck pain (Text input)	3 recommended yoga poses for neck pain, skeletal overlay near neck	As expected	Pass
2	User wants relaxing poses after work	I need yoga for stress relief (Voice input via Web Speech API)	3 calming poses, audio instructions, key point mapping on torso	As expected	Pass
3	User types a vague or unsupported query	I feel kind of ough	Prompt user for clearer input	System returns irrelevant or no poses	Fail
4	User provides a valid, clear input after initial failure	Suggest yoga for anxiety	3 relevant calming poses with benefits	As expected	Pass
5	User's webcam is on, but pose detection fails	Poses for shoulder pain	Pose suggestions + real-time skeletal overlay on shoulders	Pose suggestions appear but webcam overlay does not load	Fail
6	User refreshes webcam after failure	Give poses for shoulder pain	Pose suggestions and visible shoulder landmarks	As expected	Pass

Table 5.1 Test cases

REFERENCES

REFERENCES

- [1] N. J. van Eck and L. Waltman, "VOSviewer: A computer program for bibliometric mapping," *Scientometrics*, vol. 84, no. 2, pp. 523–538, 2010.
- [2] C. Chen, "CiteSpace II: Detecting and visualizing emerging trends and transient patterns in scientific literature," *Journal of the American Society for Information Science and Technology*, vol. 57, no. 3, pp. 359–377, 2006.
- [3] L. Waltman and N. J. van Eck, "Citation-based clustering of publications using CitNetExplorer and VOSviewer," *Scientometrics*, vol. 111, no. 2, pp. 1053–1070, 2017.
- [4] M. Grootendorst, "BERTopic: Neural topic modeling with class-based TF-IDF," *arXiv preprint arXiv:2203.05794*, 2022.
- [5] X. Li et al., "SciAgent: Tool-augmented language models for scientific reasoning," *arXiv preprint arXiv:2501.XXXX*, 2025.
- [6] L. Wang et al., "A survey on large language model-based autonomous agents," *arXiv preprint arXiv:2308.XXXX*, 2023.
- [7] Y. Huang et al., "LayoutLMv3: Pre-training for document AI with unified text and image masking," in *Proc. ACM Multimedia*, 2022, pp. 4083–4091.
- [8] S. Chai et al., "LayoutDM: Diffusion model for document layout generation," in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [9] T. Brown et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [10] OpenAlex, "OpenAlex: The open catalog of scholarly papers," [Online]. Available: <https://openalex.org>
- [11] Redis Labs, "Redis: In-memory data structure store," [Online]. Available: <https://redis.io>
- [12] ChromaDB, "Chroma: Embedding database for AI applications," [Online]. Available: <https://www.trychroma.com>
- [13] Google, "Gemini API Documentation," [Online]. Available: <https://ai.google.dev>
- [14] Meta AI, "LLaMA: Open and efficient foundation language models," *arXiv preprint*, 2023.